

LAPORAN PRAKTIKUM TEKNOLOGI IOT II



Disusun Oleh :

Muhammad Alan Al-faqih 25782074

Kelas 3C

Dosen :

Dr. Ir Septafiansyah Dwi Putra, S.T., MT.

Agus Ambarwari, S.Pd., M.Kom.

PROGRAM STUDI TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

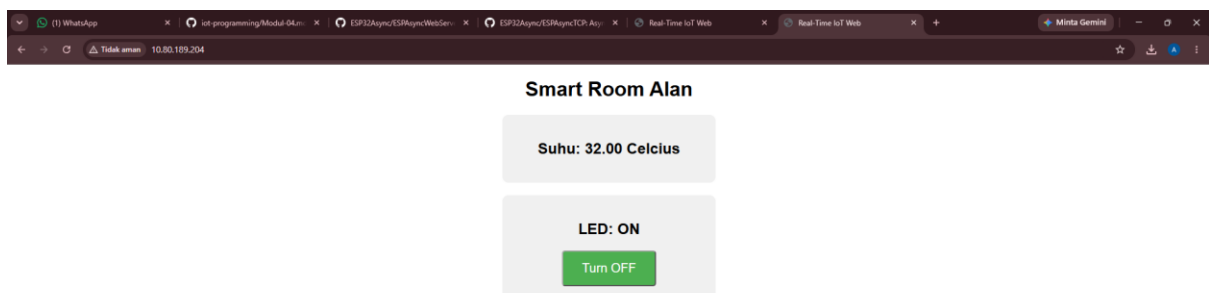
Modul 4

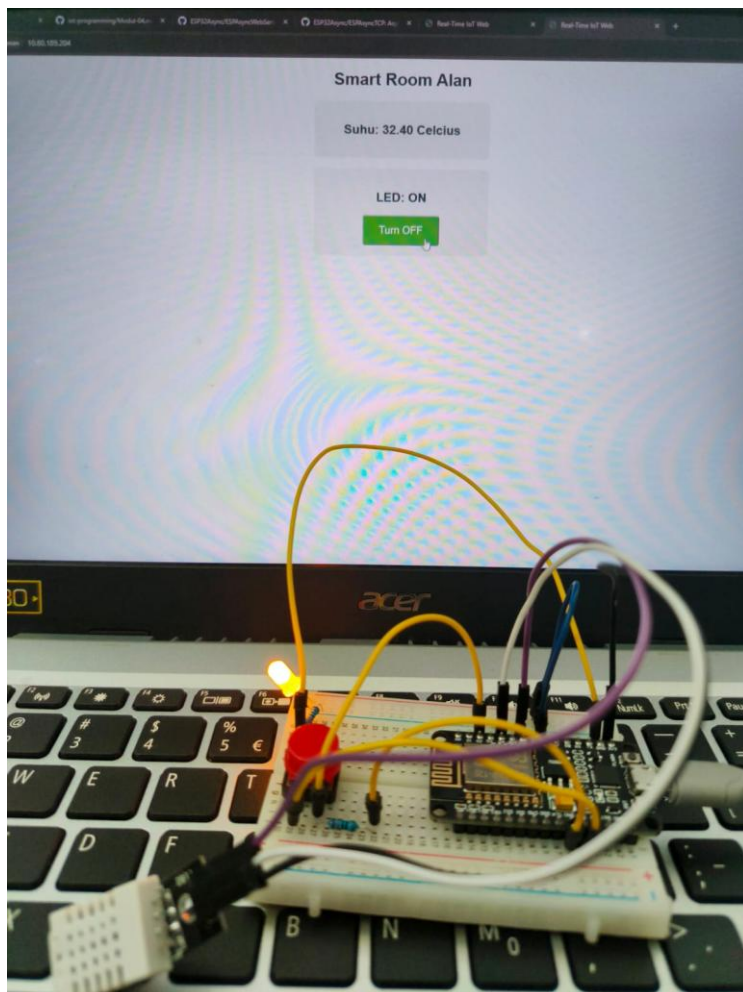
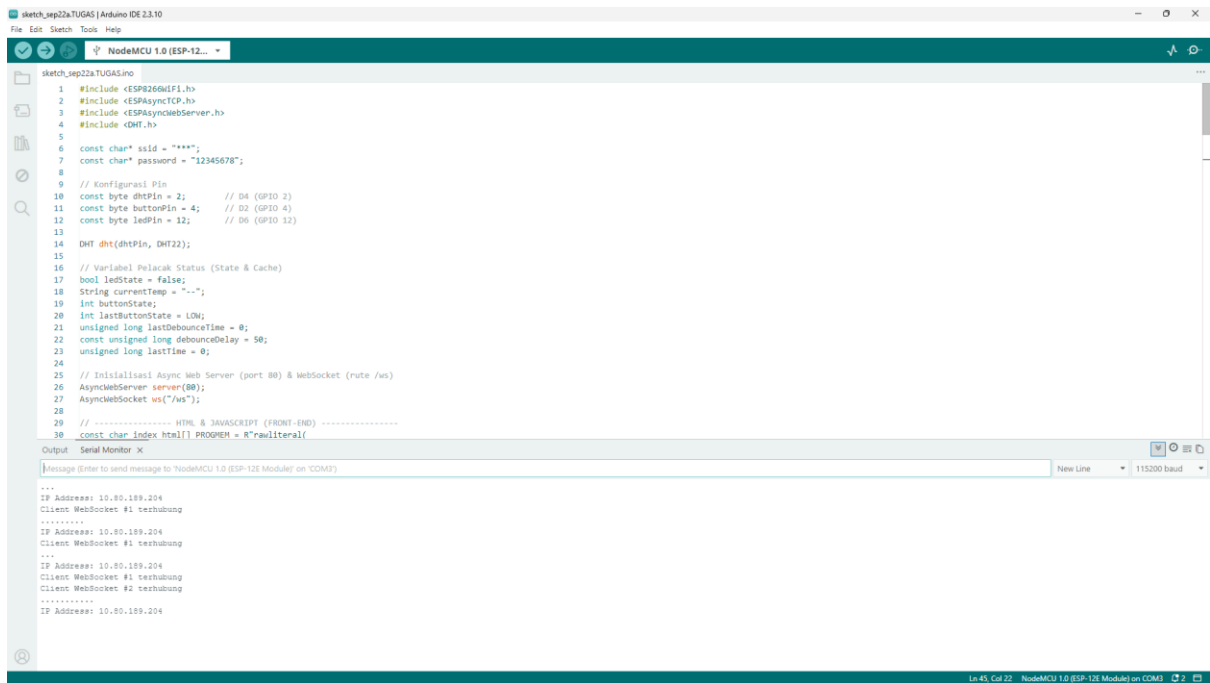
Praktikum 1: *Real-Time Dashboard & Sinkronisasi Tombol Fisik*

Skenario: Anda memiliki lampu LED (simulasi aktuator/lampu cerdas) yang bisa dihidupkan melalui sakelar fisik (*push button*) di *breadboard*, ATAU melalui tombol di antarmuka web *smartphone* Anda. Keduanya harus tersinkronisasi dua arah secara *real-time*. Jika Anda menekan tombol fisik di meja praktikum, status visual indikator di antarmuka *smartphone* harus otomatis berubah seketika, begitu pula sebaliknya jika dikontrol dari web.

Langkah Kerja Rangkaian:

1. Hubungkan kaki **Anoda** (kaki panjang) LED ke pin **D6 (GPIO 12)**. Hubungkan kaki **Katoda** (kaki pendek) LED ke salah satu kaki **Resistor 220 Ohm**, lalu hubungkan kaki resistor lainnya ke pin **GND** NodeMCU (*sama seperti rangkaian LED pada Modul 3*).
2. Hubungkan pin **DATA / OUT** DHT ke pin **D4 (GPIO 2)**. (VCC DHT ke 3V3, GND ke GND).
3. Pasang **Push Button** pada *breadboard*, melintasi parit tengah. Hubungkan salah satu kakinya ke jalur **3V3 (3.3V)**. Hubungkan kaki seberang tombol ke pin **D2 (GPIO 4)** NodeMCU. Pada titik kaki yang terhubung ke pin D2 tersebut, pasang salah satu kaki **Resistor 10k Ohm**, lalu hubungkan ujung kaki resistor lainnya ke jalur **GND** (*konfigurasi resistor pull-down eksternal seperti pada Modul 1*).





LATIHAN

Latihan 1 (Teori Jaringan): Jelaskan dengan kata-kata Anda sendiri, mengapa teknologi koneksi *WebSockets* jauh lebih hemat konsumsi data/bandwidth internet dan tidak membuat layar berkedip jika dibandingkan dengan metode *Meta Refresh HTTP* HTML yang Anda gunakan pada Tugas Praktikum Modul 3?

WebSockets lebih hemat bandwidth dibandingkan Meta Refresh HTTP karena WebSockets membuat koneksi yang tetap terbuka antara browser dan NodeMCU. Data dapat dikirim hanya ketika ada perubahan, sehingga tidak perlu melakukan request berulang kali untuk meminta data terbaru.

Pada Meta Refresh HTTP, browser harus memuat ulang seluruh halaman secara berkala. Hal tersebut menyebabkan penggunaan bandwidth lebih besar dan tampilan halaman dapat berkedip (*flickering*). Sedangkan WebSockets dapat mengirim data secara langsung (*push*) dan hanya memperbarui bagian halaman yang diperlukan tanpa melakukan *refresh* seluruh halaman.

Latihan 2 (Analisis Waktu/Timer): Apa bahaya sistematis yang akan terjadi pada komunikasi *WebSockets* (reaksi antara *browser* dan NodeMCU) jika Anda bersikeras nekat memanggil sintaks `delay(3000)` di dalam siklus fungsi `void loop()` pada draf arsitektur kompleks ini?

Jika menggunakan `delay(3000)` di dalam `void loop()`, NodeMCU akan berhenti menjalankan proses selama 3 detik. Selama waktu tersebut, proses lain seperti pemeriksaan push button dan komunikasi WebSocket tidak dapat ditangani dengan cepat.

Akibatnya, komunikasi antara browser dan NodeMCU menjadi lambat atau mengalami delay, sehingga respons tombol tidak lagi *real-time*. Oleh karena itu, pada program digunakan `millis()` sebagai metode non-blocking, sehingga NodeMCU tetap dapat menangani WebSocket, tombol, dan sensor tanpa harus berhenti selama 3 detik.

Latihan 3 (Modifikasi Front-End): Analisislah fungsi JavaScript pada blok // Merombak UI Tombol & Teks sesuai status hardware. Nama ID elemen HTML (`<span id="...">`) apa saja yang menjadi target injeksi/pengubahan teks oleh *script* yang ditulis oleh Anda?

Pada bagian JavaScript // Merombak UI Tombol & Teks sesuai status hardware, terdapat dua ID elemen HTML yang menjadi target pengubahan teks:

1. `ledStatus` → digunakan untuk mengubah teks status LED menjadi ON atau OFF.
2. `toggleBtn` → digunakan untuk mengubah tulisan tombol menjadi Turn ON atau Turn OFF, sekaligus mengubah class tampilannya menjadi `btn-on` atau `btn-off`.

Latihan 4 (Pengembangan Telemetri Sensor - Kelembapan): Pada Praktikum 1, antarmuka web hanya menyajikan data suhu (*temperature*). Modifikasilah program agar sistem dapat membaca dan menampilkan data kelembapan (*humidity*) secara *real-time* via WebSockets:

- i. **Sisi Back-End (C++):** Buat variabel penampung (misal: `String currentHum = "--";`). Baca nilai kelembapan menggunakan fungsi `dht.readHumidity()` di dalam siklus timer *non-blocking* `millis()`. Perbarui fungsi `notifyClients()` agar menyisipkan data kelembapan ke dalam paket JSON yang dikirim ke *browser* (contoh format: `{"led":"...", "suhu":"...", "hum":"..."}`).
- ii. **Sisi Front-End (HTML & JavaScript):** Tambahkan elemen kartu (`<div class="card">`) baru pada kerangka HTML sebagai penampil kelembapan (misal: `<h2>Kelembapan: <span id="humValue">--</span> %</h2>`). Kemudian, modifikasi fungsi JavaScript `onMessage(event)` agar mengekstrak data kelembapan dari objek JSON dan memperbarui teks pada elemen `humValue`.
Tuliskan baris kode modifikasi Anda (pada sisi HTML, JavaScript, dan C++)!

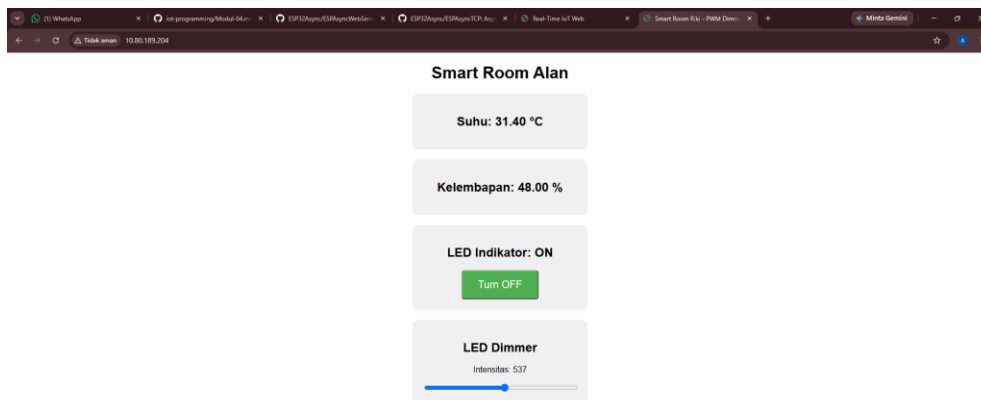
The screenshot shows the Arduino IDE environment with a sketch for a NodeMCU 1.0 (ESP-12E) module. The sketch includes code for DHT sensor initialization, variable declarations for LED state, current temperature, current humidity, and button state, and an AsyncWebServer for handling HTTP and WebSocket requests. The HTML content is embedded in the sketch, featuring a card for humidity display and a button to toggle the LED. The Serial Monitor shows the IP address 10.80.189.204 and a client connecting via WebSocket. Below the IDE, a browser window displays the 'Smart Room Alan' interface, showing the current temperature as 31.30 °C, humidity as 48.20 %, and the LED status as ON with a 'Turn OFF' button.

```
13 DHT dht(dhtPin, DHT22);
14
15 // Variabel Pelacak Status (State & Cache)
16 bool ledState = false;
17 String currentTemp = "--";
18 String currentHum = "--"; // --- Variabel penampung kelembapan baru (Latihan 4)
19 int buttonState;
20 int lastButtonState = LOW;
21 unsigned long lastDebounceTime = 0;
22 const unsigned long debounceDelay = 50;
23 unsigned long lastTime = 0;
24
25 // Inisialisasi Async Web Server (port 80) & WebSocket (route /ws)
26 AsyncWebServer server(80);
27 AsyncWebSocket ws("/ws");
28
29 // ----- HTML & JAVASCRIPT (FRONT-END) -----
30 const char index_html[] PROGMEM = R"rawliteral(
31 <DOCTYPE html>
32 <html>
33 <head>
34 <meta name="viewport" content="width=device-width, initial-scale=1">
35 <title>Real-Time IoT Web</title>
36 <style>
37 body { font-family: Arial; text-align: center; }
38 .card { background: #f0f0f0; margin: 20px auto; padding: 20px; max-width: 300px; border-radius: 10px; }
39 button { padding: 15px 30px; font-size: 20px; border-radius: 5px; cursor: pointer; color: white; }
40 .btn-on { background-color: #4CAF50; }
41 .btn-off { background-color: #f44336; }
42
```

Tugas: Sistem Kendali Intensitas Cahaya (PWM Slider) via *WebSockets*

Instruksi Pemrograman (Front-End & Back-End):

- Modifikasi desain kerangka web HTML Anda. Sisipkan elemen input *Slider* HTML. (Hint sintaksis: `<input type="range" min="0" max="1023" id="pwmSlider" onchange="sendPWM(this.value)">`)
- Buat sebuah fungsi JavaScript bernama `sendPWM(value)` yang akan mengirimkan paket *string* teks berformat khusus (misalnya "pwm,512") ke NodeMCU melalui *websocket*.
- Pada ruang fungsi C++ `handleWebSocketMessage()` di sisi server (NodeMCU), pecah (*parsing*) atau periksa string tersebut menggunakan fitur deteksi *substring* atau bahasa pemrosesan C++ lainnya.
- Jika Server NodeMCU sukses mendeteksi dan mengambil angka murninya, panggil fungsi aktuator `analogWrite(pinLED, nilaiPWM)` untuk secara elegan meredupkan atau menerangkan LED LED fisik secara seketika sesuai gesekan layar Anda!



Modul 5

Praktikum 1: Identifikasi MAC Address Perangkat NodeMCU

Sebelum membangun jaringan ESP-NOW, setiap anggota kelompok wajib membaca dan mencatat alamat fisik (*MAC Address*) dari masing-masing *board* NodeMCU.

The image shows the Arduino IDE interface with a sketch for a NodeMCU 1.0 (ESP-12E) module. The sketch is named `sketch_oct5b.ino` and contains the following code:

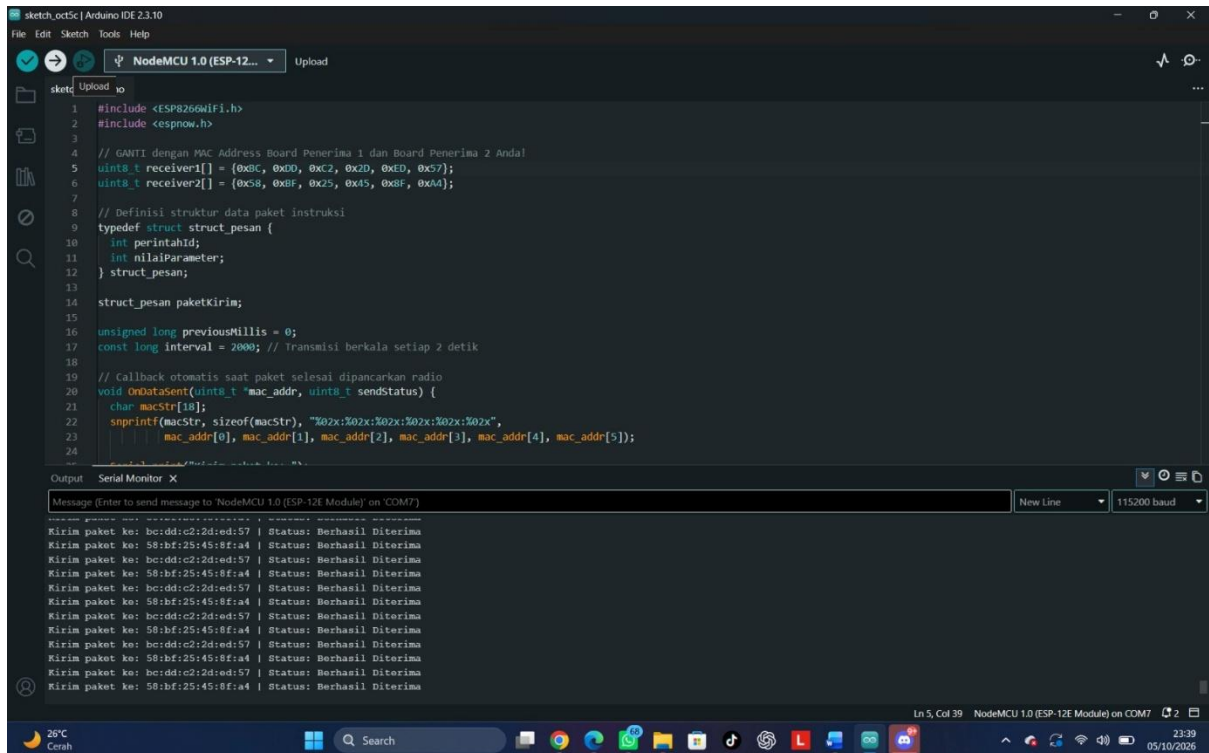
```
1 #include <ESP8266WiFi.h>
2
3 void setup() {
4   Serial.begin(115200);
5   Serial.println();
6
7   // Mengaktifkan mode Station agar MAC address antarmuka STA aktif
8   WiFi.mode(WIFI_STA);
9   WiFi.disconnect();
10
11   Serial.print("ESP8266 MAC Address: ");
12   Serial.println(WiFi.macAddress());
13 }
14
15 void loop() {
16   // Tidak ada proses berulang
17 }
```

The Serial Monitor is open, showing the output of the sketch. The message "ESP8266 MAC Address: BC1D0C121D1ED157" is displayed, indicating that the module is successfully connected to the serial port and the MAC address is being printed.

Praktikum 2: Komunikasi Nirkabel One-to-Many (ESP-NOW)

Skenario: Satu unit NodeMCU bertindak sebagai Controller / Sender, menyiarkan paket data instruksi yang sama secara langsung ke dua unit NodeMCU lain yang bertindak sebagai Slave / Receiver tanpa perantara router.

Sisi pengirim



```
1 #include <ESP8266WiFi.h>
2 #include <espnow.h>
3
4 // GANTI dengan MAC Address Board Penerima 1 dan Board Penerima 2 Anda!
5 uint8_t receiver1[] = {0x0C, 0x0D, 0xC2, 0x2D, 0xED, 0x57};
6 uint8_t receiver2[] = {0x58, 0x8F, 0x25, 0x45, 0x8F, 0xA4};
7
8 // Definisi struktur data paket Instruksi
9 typedef struct struct_pesan {
10     int perintahId;
11     int nilaiParameter;
12 } struct_pesan;
13
14 struct_pesan paketKirim;
15
16 unsigned long previousMillis = 0;
17 const long interval = 2000; // Transmisi berkala setiap 2 detik
18
19 // Callback otomatis saat paket selesai dipancarkan radio
20 void onDataSent(uint8_t *mac_addr, uint8_t sendStatus) {
21     char macStr[18];
22     sprintf(macStr, sizeof(macStr), "%02x:%02x:%02x:%02x:%02x:%02x",
23             mac_addr[0], mac_addr[1], mac_addr[2], mac_addr[3], mac_addr[4], mac_addr[5]);
24 }
25
26 void setup() {
27     Serial.begin(115200);
28     WiFi.mode(WIFI_STA);
29     WiFi.begin("ESP-NOW", "password");
30     while (WiFi.status() != WL_CONNECTED) {
31         delay(500);
32         Serial.print(".");
33     }
34     Serial.println("WiFi connected");
35 }
36
37 void loop() {
38     if (millis() - previousMillis > interval) {
39         previousMillis = millis();
40         paketKirim.perintahId = 101;
41         paketKirim.nilaiParameter = 97;
42         WiFiEspNow.send(paketKirim, receiver1, 6);
43         WiFiEspNow.send(paketKirim, receiver2, 6);
44     }
45 }
```

Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM7')

Kirim paket ke: bc:dd:c2:2d:ed:57 | Status: Berhasil Diterima
Kirim paket ke: 58:bf:25:45:8f:a4 | Status: Berhasil Diterima
Kirim paket ke: bc:dd:c2:2d:ed:57 | Status: Berhasil Diterima
Kirim paket ke: 58:bf:25:45:8f:a4 | Status: Berhasil Diterima
Kirim paket ke: bc:dd:c2:2d:ed:57 | Status: Berhasil Diterima
Kirim paket ke: 58:bf:25:45:8f:a4 | Status: Berhasil Diterima
Kirim paket ke: bc:dd:c2:2d:ed:57 | Status: Berhasil Diterima
Kirim paket ke: 58:bf:25:45:8f:a4 | Status: Berhasil Diterima
Kirim paket ke: bc:dd:c2:2d:ed:57 | Status: Berhasil Diterima
Kirim paket ke: 58:bf:25:45:8f:a4 | Status: Berhasil Diterima
Kirim paket ke: bc:dd:c2:2d:ed:57 | Status: Berhasil Diterima
Kirim paket ke: 58:bf:25:45:8f:a4 | Status: Berhasil Diterima

Sisi penerima



```
1 #include <ESP8266WiFi.h>
2 #include <espnow.h>
3
4 // Struktur data penerima (wajib identik dengan sisi pengirim)
5 typedef struct struct_pesan {
6     int perintahId;
7     int nilaiParameter;
8 } struct_pesan;
9
10 struct_pesan paketTerima;
11
12 // Callback otomatis saat antenna mendeteksi paket masuk
13 void onDataRecv(uint8_t *mac_addr, uint8_t *incomingData, uint8_t len) {
14     char macStr[18];
15     sprintf(macStr, sizeof(macStr), "%02x:%02x:%02x:%02x:%02x:%02x",
16             mac_addr[0], mac_addr[1], mac_addr[2], mac_addr[3], mac_addr[4], mac_addr[5]);
17     // Salin buffer memori radio langsung ke struktur data variabel
18     memcpy(&paketTerima, incomingData, sizeof(paketTerima));
19 }
20
21 void setup() {
22     Serial.begin(115200);
23     WiFi.mode(WIFI_STA);
24     WiFi.begin("ESP-NOW", "password");
25     while (WiFi.status() != WL_CONNECTED) {
26         delay(500);
27         Serial.print(".");
28     }
29     Serial.println("WiFi connected");
30 }
31
32 void loop() {
33     Serial.print("Paket masuk dari: ");
34     Serial.println(macStr);
35     Serial.print("Ukuran Data: ");
36     Serial.println(len);
37     Serial.print("Perintah ID: ");
38     Serial.println(paketTerima.perintahId);
39     Serial.print("Nilai Parameter: ");
40     Serial.println(paketTerima.nilaiParameter);
41 }
```

Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

Paket masuk dari: 34:94:54:64:7e:71
Ukuran Data: 8 byte
Perintah ID: 101
Nilai Parameter: 97

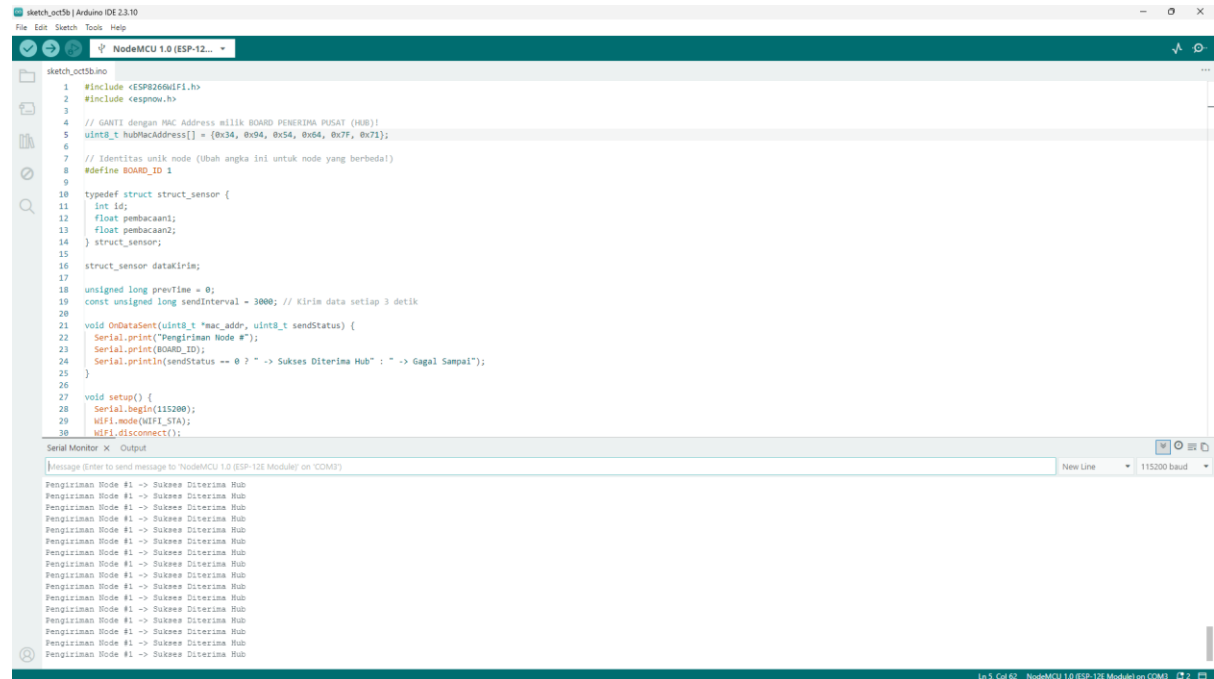
Paket masuk dari: 34:94:54:64:7e:71
Ukuran Data: 8 byte
Perintah ID: 101
Nilai Parameter: 99

Paket masuk dari: 34:94:54:64:7e:71
Ukuran Data: 8 byte
Perintah ID: 101
Nilai Parameter: 13

Praktikum 3: Pengumpulan Data Telemetri Many-to-One (ESP-NOW)

Skenario: Dua node sensor di lapangan memancarkan data pengukuran masing-masing ke sebuah node konsentrator pusat (*Aggregator Hub / Base Station*).

Sisi pengirim

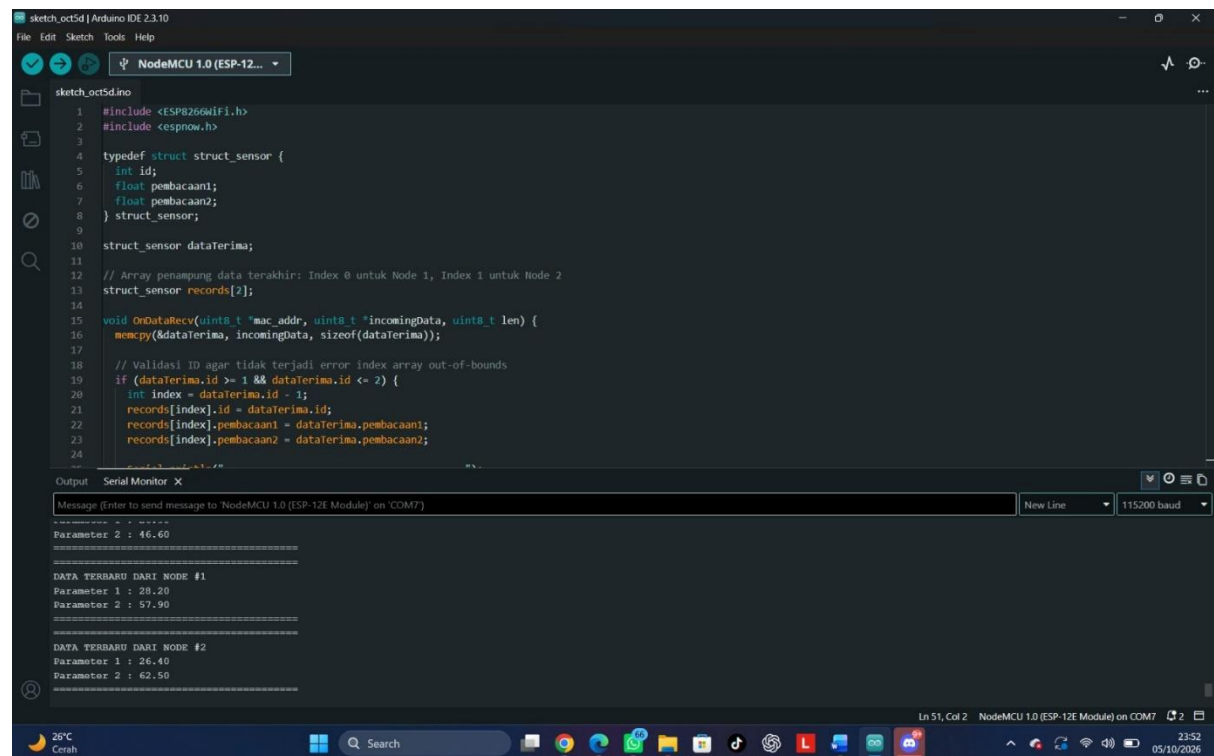


```
1 #include <ESP8266WiFi.h>
2 #include <espnow.h>
3
4 // GANTI dengan MAC Address milik BOARD PENERIMA PUSAT (HUB)
5 uint8_t hubMacAddress[] = {0x34, 0x04, 0x54, 0x64, 0x7F, 0x71};
6
7 // Identitas unik node (Ubah angka ini untuk node yang berbeda)
8 #define BOARD_ID 1
9
10 typedef struct struct_sensor {
11     int id;
12     float pembacaan1;
13     float pembacaan2;
14 } struct_sensor;
15
16 struct_sensor dataTerima;
17
18 unsigned long prevTime = 0;
19 const unsigned long sendInterval = 3000; // Kirim data setiap 3 detik
20
21 void OnDataSent(uint8_t *mac_addr, uint8_t sendStatus) {
22     Serial.print("Pengiriman Node #");
23     Serial.print(BOARD_ID);
24     Serial.println(sendStatus == 0 ? " -> Sukses Diterima Hub" : " -> Gagal Sampai");
25 }
26
27 void setup() {
28     Serial.begin(115200);
29     WiFi.mode(WIFI_STA);
30     WiFi.disconnect();
31 }
```

Serial Monitor Output:

```
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
```

Sisi penerima



```
1 #include <ESP8266WiFi.h>
2 #include <espnow.h>
3
4 typedef struct struct_sensor {
5     int id;
6     float pembacaan1;
7     float pembacaan2;
8 } struct_sensor;
9
10 struct_sensor dataTerima;
11
12 // Array penampung data terakhir: Index 0 untuk Node 1, Index 1 untuk Node 2
13 struct_sensor records[2];
14
15 void OnDataRecv(uint8_t *mac_addr, uint8_t *incomingData, uint8_t len) {
16     memcpy(&dataTerima, incomingData, sizeof(dataTerima));
17
18     // Validasi ID agar tidak terjadi error index array out-of-bounds
19     if (dataTerima.id >= 1 && dataTerima.id <= 2) {
20         int index = dataTerima.id - 1;
21         records[index].id = dataTerima.id;
22         records[index].pembacaan1 = dataTerima.pembacaan1;
23         records[index].pembacaan2 = dataTerima.pembacaan2;
24     }
25 }
```

Serial Monitor Output:

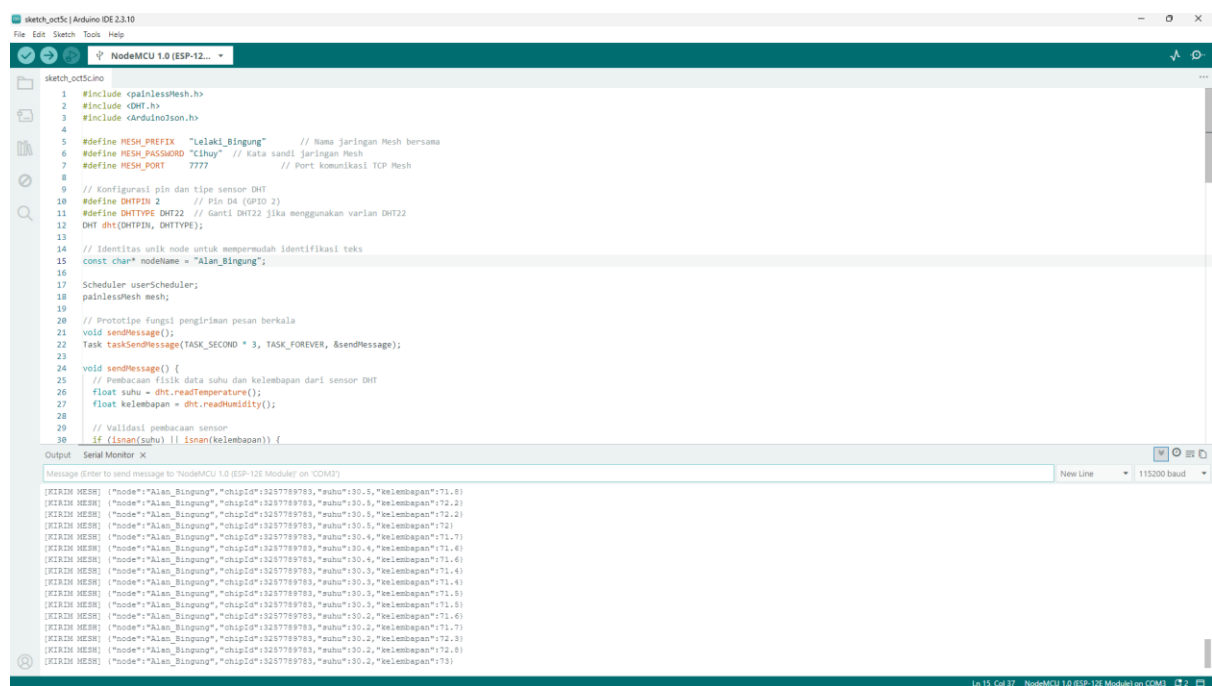
```
Parameter 2 : 46.60
=====
DATA TERBARU DARI NODE #1
Parameter 1 : 28.20
Parameter 2 : 57.90
=====
DATA TERBARU DARI NODE #2
Parameter 1 : 26.40
Parameter 2 : 62.50
=====
```

Praktikum 4: Jaringan Sensor Ad-Hoc Multi-Hop (painlessMesh)

Skenario: Seluruh board dalam kelompok dihubungkan ke sensor suhu & kelembapan riil (DHT11/DHT22) dan diintegrasikan ke dalam sebuah jaringan rantai otonom (*Mesh*). Setiap board secara berkala membaca sensor fisiknya, mengemas data telemetri ke dalam format JSON, dan menyiarkannya ke seluruh anggota jaringan tanpa perlu mengetahui alamat MAC masing-masing.

Instruksi Perangkaian Perangkat Keras:

- Pasang sensor DHT11 / DHT22 pada masing-masing board NodeMCU:
 - Pin VCC sensor ke pin 3V3 (3.3V) NodeMCU.
 - Pin GND sensor ke pin GND NodeMCU.
 - Pin DATA / OUT sensor ke pin D4 (GPIO 2) NodeMCU.



The screenshot shows the Arduino IDE interface with the 'painlessMesh' code loaded. The code defines a mesh network with a prefix 'lelaki_Bingung', a password 'Cihuy', and a port 7777. It configures a DHT22 sensor on pin D4 (GPIO 2). The code includes a scheduler to send data every 3 seconds. The Serial Monitor shows the output of the code, displaying the temperature and humidity readings for the 'lelaki_Bingung' node.

```
1 #include <painlessMesh.h>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "lelaki_Bingung" // Nama Jaringan Mesh bersama
6 #define MESH_PASSWORD "Cihuy" // Kata sandi Jaringan Mesh
7 #define MESH_PORT 7777 // Port komunikasi TCP Mesh
8
9 // Konfigurasi pin dan tipe sensor DHT
10 #define DHTPIN 2 // Pin D4 (GPIO 2)
11 #define DHTTYPE DHT22 // Ganti DHT22 jika menggunakan varian DHT22
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // Identitas unik node untuk mempermudah identifikasi teks
15 const char* nodename = "Alan_Bingung";
16
17 Scheduler userscheduler;
18 painlessMesh mesh;
19
20 // Prototipe fungsi pengiriman pesan berkala
21 void sendMessage();
22 Task taskSendMessage(TASK_SECOND * 3, TASK_FOREVER, &sendMessage);
23
24 void sendMessage() {
25     // Pembacaan fisik data suhu dan kelembapan dari sensor DHT
26     float suhu = dht.readTemperature();
27     float kelembapan = dht.readHumidity();
28
29     // Validasi pembacaan sensor
30     if (isnan(suhu) || isnan(kelembapan)) {
```

Serial Monitor Output:

```
[X] [R] [M] [H] [E] [S] [E] ("node": "Alan_Bingung", "chipId": "327789783", "suhu": 30.5, "kelembapan": 71.8)
[X] [R] [M] [H] [E] [S] [E] ("node": "Alan_Bingung", "chipId": "327789783", "suhu": 30.5, "kelembapan": 72.8)
[X] [R] [M] [H] [E] [S] [E] ("node": "Alan_Bingung", "chipId": "327789783", "suhu": 30.5, "kelembapan": 72.2)
[X] [R] [M] [H] [E] [S] [E] ("node": "Alan_Bingung", "chipId": "327789783", "suhu": 30.5, "kelembapan": 72)
[X] [R] [M] [H] [E] [S] [E] ("node": "Alan_Bingung", "chipId": "327789783", "suhu": 30.4, "kelembapan": 71.7)
[X] [R] [M] [H] [E] [S] [E] ("node": "Alan_Bingung", "chipId": "327789783", "suhu": 30.4, "kelembapan": 71.6)
[X] [R] [M] [H] [E] [S] [E] ("node": "Alan_Bingung", "chipId": "327789783", "suhu": 30.4, "kelembapan": 71.6)
[X] [R] [M] [H] [E] [S] [E] ("node": "Alan_Bingung", "chipId": "327789783", "suhu": 30.3, "kelembapan": 71.4)
[X] [R] [M] [H] [E] [S] [E] ("node": "Alan_Bingung", "chipId": "327789783", "suhu": 30.3, "kelembapan": 71.4)
[X] [R] [M] [H] [E] [S] [E] ("node": "Alan_Bingung", "chipId": "327789783", "suhu": 30.3, "kelembapan": 71.5)
[X] [R] [M] [H] [E] [S] [E] ("node": "Alan_Bingung", "chipId": "327789783", "suhu": 30.3, "kelembapan": 71.5)
[X] [R] [M] [H] [E] [S] [E] ("node": "Alan_Bingung", "chipId": "327789783", "suhu": 30.2, "kelembapan": 71.7)
[X] [R] [M] [H] [E] [S] [E] ("node": "Alan_Bingung", "chipId": "327789783", "suhu": 30.2, "kelembapan": 71.7)
[X] [R] [M] [H] [E] [S] [E] ("node": "Alan_Bingung", "chipId": "327789783", "suhu": 30.2, "kelembapan": 72.3)
[X] [R] [M] [H] [E] [S] [E] ("node": "Alan_Bingung", "chipId": "327789783", "suhu": 30.2, "kelembapan": 72.8)
[X] [R] [M] [H] [E] [S] [E] ("node": "Alan_Bingung", "chipId": "327789783", "suhu": 30.2, "kelembapan": 73)
```

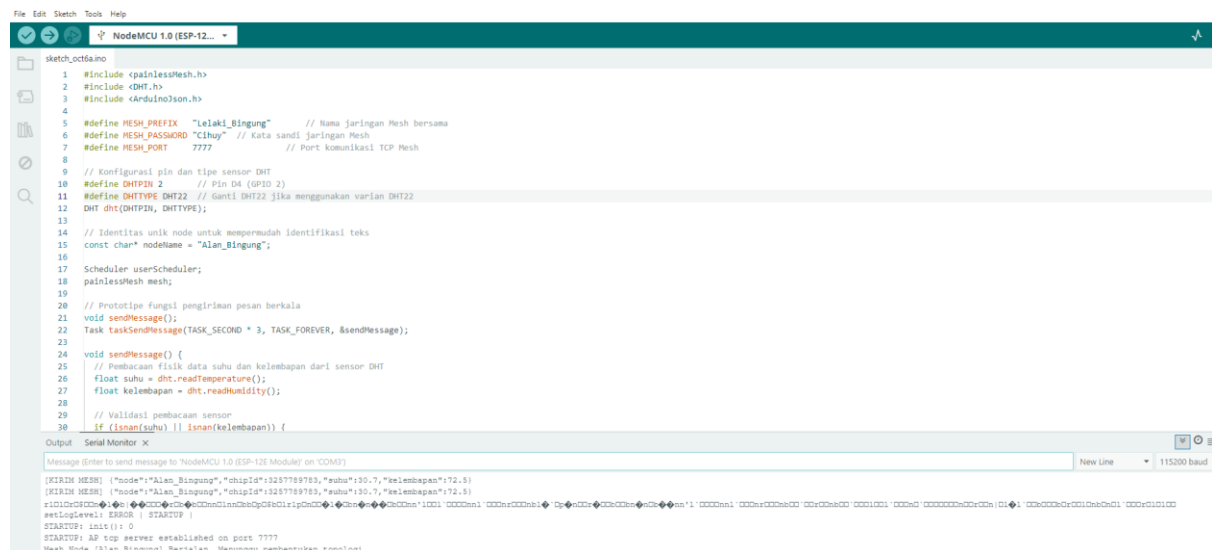
Latihan 1

[illegible]

Latihan 1 (Eksperimen Deteksi ACK & Pemutusan Simpul - Praktikum 2): Saat program Praktikum 2 (ESP-NOW One-to-Many) sedang berjalan normal dan mencetak status "Berhasil Diterima" (sendStatus == 0), lakukan eksperimen pemutusan simpul:

- i. Cabut kabel USB salah satu board Penerima (misal NodeMCU 2) dari sumber daya.
- ii. Amati perubahan log pada Serial Monitor NodeMCU Controller pengirim! Apakah status pengiriman untuk MAC address board yang dicabut berganti menjadi "Gagal (Tidak Terjangkau)"?
- iii. Jelaskan bagaimana protokol ESP-NOW pada lapisan *Layer 2* MAC dapat mendeteksi kegagalan transmisi ke perangkat fisik tersebut meskipun sistem tidak menggunakan koneksi TCP!

Latihan 2



- **Latihan 2 (Eksperimen Dinamika Topologi & Self-Healing - Praktikum 4):** Jalankan jaringan painlessMesh pada Praktikum 4 hingga seluruh board saling terhubung dan mencetak pesan --> Koneksi Baru Terdeteksi! Node ID:

- i. Tekan tombol fisik **RST** atau cabut daya salah satu board Node. Amati Serial Monitor pada board lainnya yang masih menyala. Berapa detik waktu yang dibutuhkan oleh *callback* `changedConnectionCallback()` untuk mendeteksi bahwa salah satu simpul telah keluar dari rantai mesh? **0.5 detik**
- ii. Sambungkan kembali daya board tersebut. Berapa lama waktu yang dibutuhkan hingga node tersebut kembali terdaftar secara otonom ke dalam topologi mesh tanpa Anda mengonfigurasi ulang jaringan? Catat hasil observasi waktu tersebut!

0.5 detik

Latihan 3

